

PROGRAMMAZIONE II

TIPI GENERICI

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

A.A. 2025/2026

STESSO CONTENITORE, CONTENUTO DIVERSO



POLIMORFISMO PARAMETRICO

Metodi e parametri possono avere un tipo generico, usando **variabili di tipo**

- ▶ Scrivere codice indipendente dal tipo di dati gestiti

Esempio banale, la funzione identità $\lambda x . x$

- ▶ Se x ha tipo `int`, $\lambda x . x$ restituisce lo stesso `int` fornito in input
- ▶ Se x ha tipo `String`, $\lambda x . x$ restituisce la stessa `String` fornita in input

POLIMORFISMO PARAMETRICO (CONT.)

Come possiamo implementare un metodo identità in Java?

- ▶ Ingenuamente, usare `Object` come segnaposto
- ▶ `Object identity(Object obj) { return obj; }`

Scomodo e poco elegante, possiamo usare i **tipi generici**

// noti come generici in Java

- ▶ Qui `<T>` è la dichiarazione del parametro di tipo per il metodo
- ▶ Quando il metodo viene chiamato, si deve fornire un tipo effettivo per `T`
- ▶ Qualsiasi tipo è valido per `T`, tranne i tipi primitivi

```
<T> T identity(T x) { return x; }
```

CLASSE PARAMETRICA SENZA GENERICI

Classe che rappresenta coppie di valori, senza generici

- Usare `Object` per i valori della coppia

```
public class Pair {                                1
    private Object first, second;                  2

    public Pair(Object first, Object second) {      3
        this.first = first;                        4
        this.second = second;                      5
    }                                                6
                                                    7
    public Object getFirst() { return first; }      8
    public Object getSecond() { return second; }   9
}                                                  10
                                                    11
```

CLASSE PARAMETRICA SENZA GENERICI (CONT.)

È possibile usare la classe con tipi diversi

```
Pair p = new Pair("one", "two");
```

1

```
Pair q = new Pair(1, 2); // auto-boxing verso Integer
```

2

Ma bisogna fare molta attenzione ai cast

```
String first = (String) p.first();
```

1

```
String second = (String) q.second(); // ClassCastException a runtime
```

2

CLASSE PARAMETRICA CON I GENERICI

Classe che rappresenta coppie di valori, con i generici

```
public class Pair<T> { // dichiarazione del parametro di tipo della classe 1
    private T first, second; 2
    3
    public Pair(T first, T second) { 4
        this.first = first; 5
        this.second = second; 6
    } 7
    8
    public T getFirst() { return first; } // dichiarazione parametro di tipo 9
    public T getSecond() { return second; } // non necessaria qui 10
} 11
```


CLASSE PARAMETRICA CON I GENERICI (CONT.)

È possibile usare la classe con tipi diversi

```
Pair<String> p = new Pair<String>("one", "two");  
Pair<Integer> q = new Pair<Integer>(1, 2);
```

1

2

Ma non sono necessari i cast

```
String first = p.first();  
String second = q.second(); // errore di compilazione -- tipo incompatibile
```

1

2

DICHIARAZIONE DI TIPI GENERICI

Le dichiarazioni di tipi generici possono contenere più di un parametro di tipo

- Possiamo creare riferimenti, metodi, classi e interfacce generiche

$\langle T_1, T_2, \dots, T_k \rangle$

Convenzioni sui nomi

- Lettere maiuscole
- Di solito T (ype), E (lement), K (ey), V (alue)

INTERFACCIA COMPARABLE CON I GENERICI

L'interfaccia `java.lang.Comparable<T>` contiene il metodo

```
int compareTo(T obj);
```

```
public class Student extends Person implements Comparable<Student> { 1
    ... 2
    @Override 3
    public int compareTo(Student other) { // codice più semplice e sicuro 4
        return this.matricola - other.matricola; 5
    } 6
} 7
```

INTERFACCE ITERABLE E ITERATOR CON I GENERICI

L'interfaccia `java.lang.Iterable<E>` contiene il metodo

```
Iterator<E> iterator();
```

L'interfaccia `java.util.Iterator<E>` contiene i metodi

```
boolean hasNext(); e E next();
```

INTERFACCE ITERABLE E ITERATOR CON I GENERICI (CONT.)

```
import java.util.Iterator;
import java.lang.Iterable;

class Classroom implements Iterable<Student> {
    private Student[] students;

    Classroom(Student[] students) {
        this.students = students;
    }

    @Override
    public Iterator<Student> iterator() {
        return new CRIterator(students);
    }
    ...
}
```

```
...
class CRIterator implements Iterator<Student> {
    private int next = 0;

    @Override
    public boolean hasNext() {
        return next < arr.length;
    }

    @Override
    public Student next() {
        return new Student(arr[next++]);
    }
}
}
```

```
Student[] arrS = {new Student("Sam", 456), new Student("Paul", 123)};
ClassRoom classRoom = new ClassRoom
for (Student student : classRoom) // ora il codice è davvero più elegante
    System.out.println(Student.toStri
```

OPERATORE DIAMOND

Il parametro di tipo del riferimento deve coincidere con quello usato all'istanziamento

```
Pair<Integer> intPair = new Pair<Integer>(2, 4);
```

A partire da Java 7, il compilatore può dedurre il parametro di tipo all'istanziamento tramite <>

```
Pair<Integer> intPair = new Pair<>(2, 4);
```

PARAMETRO DI TIPO NON VINCOLATO

Per impostazione predefinita, T è un segnaposto per un tipo non vincolato

- ▶ Non ci sono vincoli sul tipo che possiamo sostituire a T
- ▶ Il compilatore assume in sicurezza che T sia sostituito da `Object`

```
public class Pair<T> { // qui vorremmo usare Person      1
    private T first, second;                                2

    public Pair(T first, T second) {                        3
        this.first = first;                                4
        this.second = second;                              5
    }                                                        6
                                                            7
    public T getFirst() { return first.getName()           8
    public T getSecond() { return second; }                9
    }                                                        10
                                                            11
```

// errore di compilazione
// metodo non trovato per T

PARAMETRO DI TIPO VINCOLATO

Possiamo definire vincoli sui tipi che possiamo sostituire a T

Specificando un **limite superiore** (upper bound)

- Possiamo istanziare T con $S1, S2, \dots, Sk$ o una delle loro sottoclassi

```
class Clazz<T extends S1 & S2 & ... & Sk>
```

Specificando un **limite inferiore** (lower bound)

- Possiamo istanziare T con S o una delle sue superclassi

```
class Clazz<T super S>
```


PARAMETRO DI TIPO VINCOLATO (CONT.)

Se **T** è vincolato, il compilatore può dedurre i possibili metodi invocabili

```
1 public class Pair<T extends Person> { // qui vogliamo usare Person
2     private T first, second;
3
4     public Pair(T first, T second) {
5         this.first = first;
6         this.second = second;
7     }
8
9     public T getFirst() { return first.getName(); } // nessun errore
10    public T getSecond() { return second; } // T estende Person
11 }
```

GENERICI ED EREDITARIETÀ



SOTTOTIPAZIONE GENERICA

Bisogna prestare attenzione quando si applica il principio di sostituzione

- ▶ Dato che `Student` è un sottotipo di `Person`
- ▶ Tuttavia `Pair<Student>` **non è** un sottotipo di `Pair<Person>`

```
Student s = new Student("Sam", 123);           1
Person p = s; // ok                             2
                                                3
Pair<Student> pairS = new Pair<>(s, new Student("Paul", 456)); 4
Pair<Person> pairP = pairS; // errore di compilazione 5
```

In generale, `Clazz<T>` e `Clazz<S>` non sono confrontabili in termini di sottotipazione

SOTTOTIPAZIONE DEGLI ARRAY

Bisogna prestare attenzione quando si applica il principio di sostituzione

- ▶ Dato che `Student` è un sottotipo di `Person`
- ▶ E `Student[]` è un sottotipo di `Person[]`

```
Student s = new Student("Sam", 123);
```

```
Person p = s; // ok
```

```
Student[] arrS = {sam, new Student("Paul", 456)};
```

```
Person[] arrP = arrS; // ok
```

1

2

3

4

5

WILDCARD

Con le **wildcard** possiamo esprimere (la mancanza di) vincoli nell'uso di tipi generici

- ▶ `Clazz<?>` indica una `Clazz` sconosciuta (non vincolata)
- ▶ Il tipo di `?` è sconosciuto, quindi viene trattato come `Object`

```
Student s = new Student("Sam", 123);           1
Person p = s; // ok                             2
                                                3
Pair<Student> pairS = new Pair<>(s, new Student("Paul", 456)); 4
Pair<?> pairP = pairS; // ok                     5
```

... ma non possiamo invocare metodi di `Person`, come `pairP.getFirst().getName()`

WILDCARD (CONT.)

Con le **wildcard** possiamo esprimere (la mancanza di) vincoli nell'uso di tipi generici

- ▶ `Clazz<? extends T>` indica una `Clazz` di `T` o una sua sottoclasse
- ▶ `Clazz<? super T>` indica una `Clazz` di `T` o una sua superclasse

```
<T extends Person> void printPair(Pair<T> p) {  
    System.out.println(p.getFirst().getName());  
    System.out.println(p.getSecond().getName());  
}
```

```
void printPair(Pair<? extends Person> p) {  
    System.out.println(p.getFirst().getName());  
    System.out.println(p.getSecond().getName());  
}
```

Analogo ai parametri di tipo, ma meno prolisso

- ▶ Non sono necessarie dichiarazioni di parametri di tipo per classi e metodi

WILDCARD (CONT.)

Con le **wildcard** possiamo esprimere (la mancanza di) vincoli nell'uso di tipi generici

- ▶ `Clazz<? extends T>` indica una `Clazz` di `T` o una sua sottoclasse
- ▶ `Clazz<? super T>` indica una `Clazz` di `T` o una sua superclasse

```
Student s = new Student("Sam", 123);           1
Person p = s; // ok                             2
                                                3
Pair<Student> pairS = new Pair<>(sam, new Student("Paul", 456)); 4
Pair<? extends Person> pairP = pairS; // ok      5
```

Ora possiamo invocare metodi di `Person`, come `pairP.getFirst().getName()`



DOMANDE E RISPOSTE

